

CHRISTALIN MIRRORS

Billing System Plan & Pine Labs Integration

Point-of-Sale, GST invoicing, bill archival, and card-terminal payment integration across branches.

Prepared: 30 July 2026 · Version 1.0

Scope: Bengaluru & Kalaburagi branches (multi-branch ready)

Audience: Owner / Operations / Pine Labs onboarding

Contents

1	Executive summary
2	Current system (what exists today)
3	Billing workflow — end to end
4	Invoice data model
5	GST handling (5% · CGST + SGST split)
6	Roles & multi-branch scoping
7	Auto-save bills to a folder (receptionist)
8	Printing
9	Multi-branch / multi-terminal architecture
10	Implementation roadmap (phased)
11	Open decisions
A	Pine Labs — Prerequisites, Checklist & Questions to Ask

1 • Executive summary

This document describes the salon's billing system — how it works today, the two features being added (automatic bill archival for the front desk, and card-terminal payments via Pine Labs), and exactly what must be obtained from Pine Labs before their integration can be built.

The billing module is a web-based Point-of-Sale used by the front desk. It creates GST-compliant invoices, records payment, shares bills over WhatsApp, and is scoped per branch. Two enhancements are planned:

- **Bill archival:** automatically save each finalised bill as a PDF into a folder on the reception computer for later reprinting.
- **Pine Labs card payments:** send the bill amount directly to the branch's card machine so the invoice is marked paid automatically, across multiple branches and multiple terminals.

Key principle for card payments: card data is handled entirely by the Pine Labs terminal — it never enters our software. This keeps the salon's PCI-DSS compliance scope minimal.

2 • Current system (what exists today)

Capability	Status & notes
Point-of-Sale (create bill)	Live — branch, client, stylist, line items (services & retail products), discount, GST, payment method.
Client registration	Live — search existing client or register a new one during billing.
GST invoice	Live — subtotal, discount, GST, grand total; live receipt preview.
GST split (CGST + SGST)	Live — GST line now shown as CGST + SGST on preview, saved invoice, and WhatsApp text.
Payment methods	Live — Cash / UPI / Card recorded manually (cash gives change calculation).
WhatsApp bill share	Live — sends a formatted bill summary to the client.
Print bill	Live — opens the invoice and uses the browser print dialog.
Multi-branch	Live — data scoped to the logged-in user's branch; owner sees all.
Auto-save to folder	Planned — see §7.
Pine Labs terminal payment	Planned — see §9 & Appendix A.

Data today is held in the application's front-end store. A backend service exists in the project and will host secure integrations (e.g. Pine Labs credentials) — see §9.

3 • Billing workflow — end to end

1. **Branch** — auto-set to the receptionist's branch (owner can choose).
2. **Client** — search & select an existing client, or register a new one on the spot.

3. **Stylist / therapist** — assign who performed the service.
4. **Items** — add services and/or retail products; set quantity and price.
5. **Discount** — percentage or flat, applied to the subtotal.
6. **GST** — applied on the taxable amount and shown as CGST + SGST (\$5).
7. **Payment** — Cash / UPI / Card. (Future: "Charge on Pine Labs terminal" — \$9.)
8. **Finalise** — invoice saved with a unique number; success screen offers Print & WhatsApp. (Future: PDF auto-archived to folder — \$7.)

4 • Invoice data model

Field	Meaning
<code>invoiceNumber</code>	Unique human-readable ID (e.g. CM-INV-0001) — also used as the payment reference.
<code>clientName</code> / <code>clientEmail</code> / <code>clientPhone</code>	Customer details (phone used for call & WhatsApp).
<code>items[]</code>	Each line: service/product, quantity, unit price, total.
<code>subtotal</code>	Sum of line items before discount/tax.
<code>discountPercent</code> / <code>discountAmount</code>	Discount applied.
<code>taxPercent</code> / <code>taxAmount</code>	GST rate and value (displayed as CGST + SGST).
<code>total</code> / <code>amountPaid</code>	Grand total and amount collected.
<code>paymentMethod</code> / <code>status</code>	cash / upi / card; paid / sent / etc.
<code>branch</code> / <code>stylist</code> / <code>date</code>	Branch, staff, and date of the bill.

Future additions for the enhancements: `clientGSTIN` (optional, for B2B invoices), and Pine Labs fields such as `terminalId` and `pgTransactionRef`.

5 · GST handling (5% · CGST + SGST split)

Salon GST is configured at **5%**. Because services are consumed in-state (intra-state supply), the GST is always split equally into two halves that are reported to different governments:

Component	Rate	On a ₹400 service	Goes to
CGST	2.5%	₹10	Central government
SGST	2.5%	₹10	State government
Total GST	5%	₹20	Grand total ₹420

- The split is **display-only** — totals and stored values are unchanged; the single GST line is simply shown as two lines on the receipt, saved invoice, and WhatsApp text.
- Halves are computed so the two rounded amounts always sum exactly to the total GST (no ₹1 rounding gaps on odd amounts).
- Inter-state sales would instead use a single **IGST 5%** line — not applicable to a walk-in salon.

To confirm with your accountant: the correct GST rate for the salon's services (5% is configured; hairdressing/beauty services are often 18%), and whether your GSTIN, SAC codes, and CGST/SGST breakup must also be printed on the invoice for full compliance.

6 · Roles & multi-branch scoping

Role	Branch access	Billing rights
Owner	All branches	Full visibility, all-branch reporting.
Manager	Own branch only	Billing, staff, attendance for their branch.
Receptionist	Own branch only	Front-desk billing, clients, invoices.

- Appointments, staff, invoices and revenue are **scoped to the user's branch**; the owner sees everything.
- **Clients are shared across branches** — a client who visits another branch is still visible (they may return to either location).
- The planned **auto-save-to-folder** feature is **receptionist-only** (\$7).

7 · Auto-save bills to a folder (receptionist)

Goal: after a bill is finalised, a PDF is automatically stored in a folder on the reception PC so it can be reprinted later.

Browser limitation: a website cannot silently write to an arbitrary path like `C:\Bills\`. A normal download always lands in the browser's Downloads folder and JavaScript cannot choose the path. The two viable approaches below work around this.

Option A — File System Access API Recommended to start

- Receptionist picks the "Bills" folder **once**; the app remembers that folder.
- Each finalised bill is written there automatically as a PDF, e.g. `CM-INV-0001_ArjunDesai_2026-07-30.pdf`.
- Bills can be opened and printed from that folder anytime.

Caveats: Chromium browsers only (Chrome / Edge); requires HTTPS; after a full browser restart the receptionist confirms folder access with one click at the start of the day, then it is silent for the session.

Option B — Local agent / desktop wrapper Bulletproof, more work

- A small local program on the reception PC receives the PDF and writes it to any fixed path — and can **auto-print** to the connected printer with zero clicks.
- No browser limitations; true hands-off archival and printing.

Work required either way

- Add PDF generation from the existing invoice layout.
- Gate the feature to the **receptionist** role.
- Consistent file naming: invoice no. + client + date; optionally sub-foldered by branch or date.

Recommendation: ship Option A first (covers "auto-save to a folder for later printing" with minimal effort). Move to Option B later if fully automatic, zero-click printing is required.

8 · Printing

- Today: the invoice opens and prints via the browser's print dialog.
- With Option A (\$7): print any archived PDF from the folder.
- With Option B (\$7): optional automatic printing at finalisation.
- Thermal receipt printers (80mm) can be supported later with a receipt-sized layout if desired.

9 · Multi-branch / multi-terminal architecture (Pine Labs)

Pine Labs uses a three-level hierarchy that maps directly onto the salon's structure:

```
Merchant (Christalin Mirrors – one company account)
├── Store (one per branch: Bengaluru, Kalaburagi)
│   └── Terminal (each physical card machine – identified by IMEI / serial)
```

When the software initiates a payment it names the **target terminal** (IMEI, and/or Store ID) so the request reaches the *correct physical machine* — not every terminal in the company.

How it fits our app

1. **Configuration (owner, once):** for each branch, store its **Store ID** and a list of **terminals** (IMEI + friendly label, e.g. "Front Desk 1").
2. **At billing:** the receptionist's branch auto-filters to that branch's terminals.
 - One machine → auto-selected.
 - Several machines → a "send to terminal" choice, or pin each reception station to its own terminal (zero choosing).
3. **Transaction:** backend sends *amount + invoice number (reference) + terminal IMEI/Store ID* → Pine Labs routes it → customer pays on that machine → result returns → invoice marked **paid** on the correct branch.

Why a backend is required

- Pine Labs credentials (merchant secret/token) must live **server-side**, never in the browser.
- The backend initiates the charge, receives the callback/poll result, and reconciles it to the invoice.
- Card data stays on the terminal → minimal PCI scope for the salon.

App deliverables (once credentials are available): (a) a *Settings → Payment Terminals* screen to map each branch to its Store ID and terminals; (b) a "*Charge on terminal*" button in billing that auto-marks the invoice paid on success.

10 · Implementation roadmap (phased)

Phase	Deliverable	Depends on
1 	GST CGST + SGST split (done)	—
2	PDF auto-save to folder (Option A), receptionist-only	Confirm reception browser (Chrome/Edge)
3	Settings → Payment Terminals (branch → store → terminal mapping)	Pine Labs Store IDs & terminal IMEIs
4	Backend Pine Labs endpoint (initiate + reconcile)	Pine Labs API docs + sandbox credentials
5	"Charge on terminal" in billing; auto-mark paid	Phase 3 & 4
6	Optional: refunds/voids, daily settlement reconciliation, thermal receipts, auto-print (Option B)	Phase 5

Phases 3 can begin now (it is our own configuration data). Phases 4–5 need Pine Labs onboarding — see Appendix A.

11 • Open decisions

-
- ☐ Confirm correct **GST rate** (5% configured) and whether GSTIN/SAC must print on the invoice — with accountant.
 - ☐ Choose folder-archival approach: **Option A** (recommended) vs Option B; confirm reception browser is Chrome/Edge.
 - ☐ Do you want an optional **client GSTIN field** for B2B invoices?
 - ☐ Number of Pine Labs **terminals per branch**, and whether each reception station is pinned to a terminal.
 - ☐ Whether invoices should also live in the **backend/database** (recommended before Pine Labs reconciliation).

A · Pine Labs — Prerequisites, Checklist & Questions to Ask

Pine Labs integration cannot be built until the items below are obtained from your Pine Labs relationship manager. Use this appendix as the brief to send them.

A.1 · Things needed before development (prerequisites)

- ☐ An active **Pine Labs merchant account** for the company.
- ☐ Confirmation of the integration product: **Cloud-based POS integration** (Plutus Smart cloud API) suitable for multi-branch billing software.
- ☐ **Merchant ID** (company-level).
- ☐ **Store IDs** — one per branch (Bengaluru, Kalaburagi).
- ☐ **Terminal IMEIs / serials** — for every card machine, mapped to its branch.
- ☐ **Client ID + security token/secret** for API authentication.
- ☐ **Sandbox / UAT credentials and a test terminal** for integration testing.
- ☐ Full **API / integration documentation** (request/response formats, status codes).
- ☐ A named **technical / integration support contact** at Pine Labs.
- ☐ Any **IP-whitelisting or request-signing** requirements for production.

A.2 · Questions to ask Pine Labs

Account & structure

1. We are one company with **multiple branches and multiple card machines per branch**. Can we have **one Merchant ID → one Store per branch → multiple Terminals per store**? How are Store IDs and Terminal IMEIs provisioned and listed?
2. Which integration product suits centralised multi-branch billing — **Cloud (Plutus Smart)** or LAN/local integration? What are the pros/cons for our case?

Credentials & environments

3. Exact credentials required for **sandbox and production**: Merchant ID, Store ID(s), Client ID, security token/secret, Terminal IMEIs.
4. Please provide **API documentation** and a **sandbox/UAT terminal** to test before go-live.

Transaction API

5. How do we **target a specific terminal** in a payment request — by IMEI, Store ID, or both?
6. Is the result delivered by **callback/webhook** or do we **poll** for status? What are the status codes, the **timeout**, and how do we handle a customer cancelling on the machine?
7. Which field carries **our invoice/bill number** (merchant reference) so we can match a payment back to the right bill?
8. **Idempotency**: how do we prevent a **double charge** if a request is retried after a network drop (unique reference handling)?

Operations, reconciliation & refunds

9. API for **refunds, partial refunds, and voids** (e.g. a wrong charge).

- L0. **Settlement / MIS reports API** to reconcile takings **per branch** daily.
- 11. Which **payment modes** can be enabled on the terminals (card, UPI, wallets, EMI)?
- L2. Adding **new terminals** later — self-service portal or via Pine Labs? Turnaround time?
- L3. Production **security requirements** (IP whitelisting, request signing) and confirmation the **terminal handles all card data** (so our software stays out of PCI scope).
- L4. **Support & SLA** — escalation contact and response times for terminal/API issues.

A.3 · Information to hand back to the development team

Once Pine Labs responds, collect these so integration can begin:

Item	Value (fill in)
Integration type (Cloud / LAN)	
Merchant ID	
Store ID — Bengaluru	
Store ID — Kalaburagi	
Terminal IMEIs (per branch)	
Client ID / security token	
Sandbox credentials + test terminal	
Result mechanism (callback / poll)	
Merchant-reference field name	
Support contact	

After A.3 is filled in, development proceeds with Phase 3 (terminal configuration UI) → Phase 4 (backend Pine Labs endpoint) → Phase 5 ("Charge on terminal" + auto-mark paid).